

Remember: UML Use Case Diagrams

Modeling requirements during systems analysis

Use Case (UC)

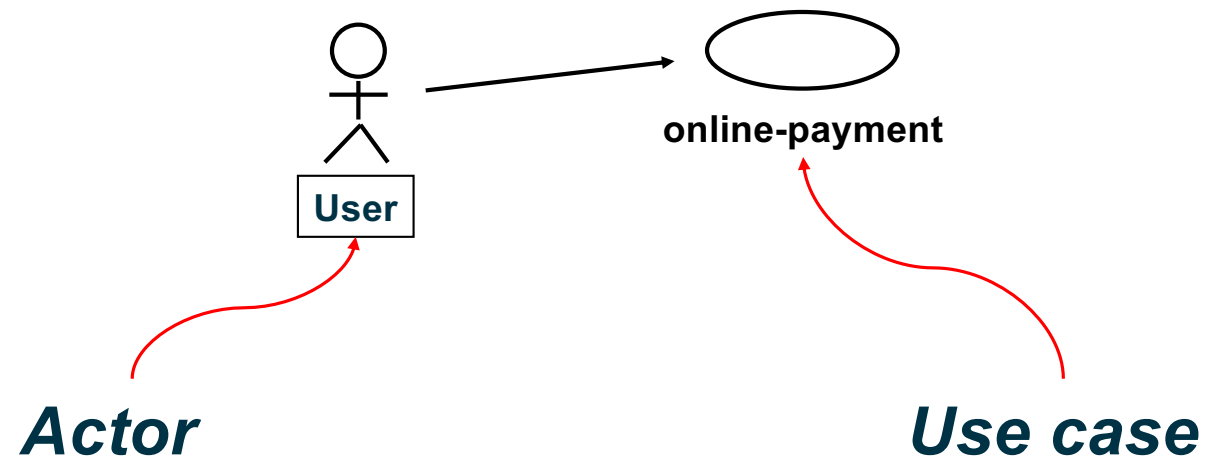
- Used to capture functional requirements and/or document the system's functionality
- No implementation details, it is about the **End-User-Perspective**
- Dr. Ivar Jacobson (Swedish computer scientist) introduced use-case philosophy for his work on large telecommunication systems
- Each UC is a complete series of events, from the point of an actor
- „**Who**“ (actor) does „**what**“ (interaction) with the system for what „purpose“ (goal)?
- Actor: something (system, device) / someone existing outside the system to achieve some goal in dialog with the system
- Use Cases can be done with **UML** (Unified Modeling Language)



Use Case Diagram Elements

- The Use Case Diagram is used to identify the primary elements and processes that form the system. The primary elements are termed as "actors" and the processes are called "use cases."
- The Use Case Diagram shows which actors interact with each use case.
- UML Use Case Diagrams (UCDs) can be used to describe the functionality of a system, they capture the functional aspects of the system.
- UCDs have only 4 major elements:
 - The **actors** that the system you are describing interacts with,
 - the **system** (boundary) itself,
 - the **use cases**, or services, that the system knows how to perform,
 - the lines that represent **relationships between these elements**.

Use Case Diagram (UCD)



Use Case

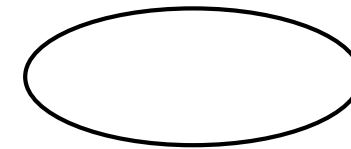


A use case is a **distinct business functionality** (process) or behavior pattern of the system (a flow of action within the application area)

Every use case is a sequence of related activities carried out by an actor together with the system.

A use case represents **some action the user might perform in order to complete a task** (from an external point of view).

• .

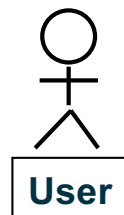


online-payment

Actor



- An actor is an **external interface** of the system (mostly a user, but external systems are possible, too). An actor is any entity that performs certain roles in a given system
- Actors **activate flows of actions** in a system and influence them



Use Case Documentation

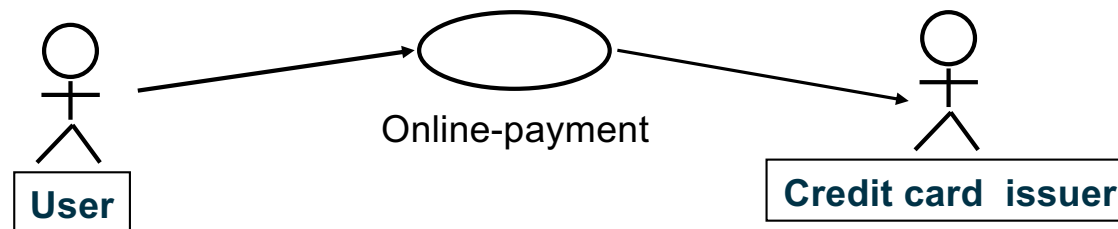


- For each use case the flow of actions is described from the actor's view.
- Special emphasis is on the interface between the actor and the use case.
For example:
 - How is a use case initiated and how is it terminated
 - Control flow in normal operations
 - Exceptions: When do they occur, what is the reaction

Use Case Diagram

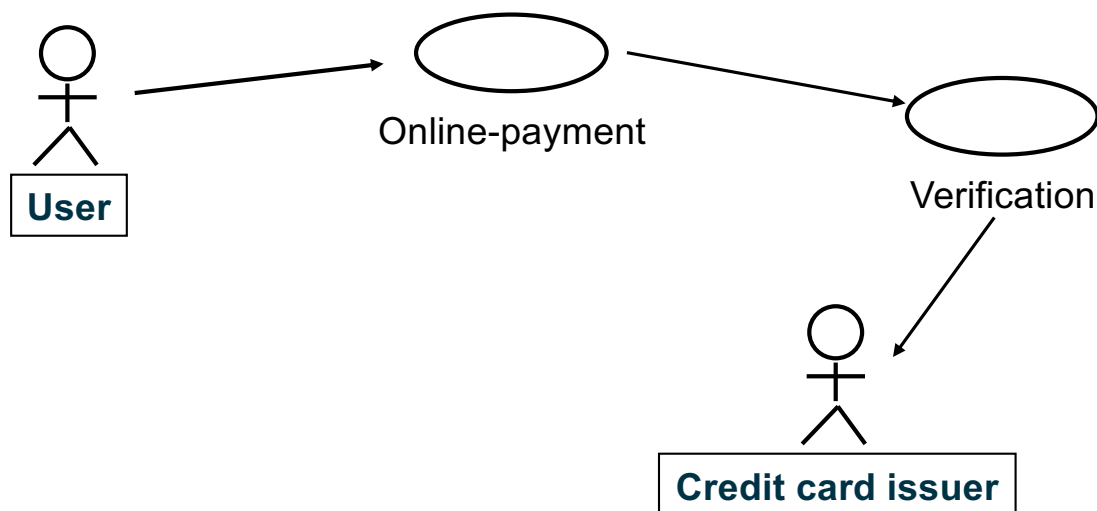


- A use case can have several Actors attached to it:



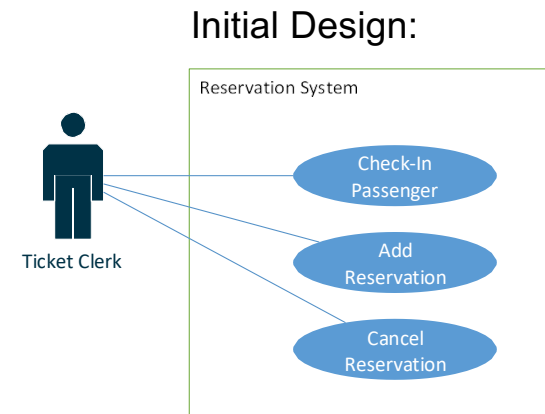
Use Case Diagram

- In a use case diagram several use cases can appear:



Example - Airline Reservation System

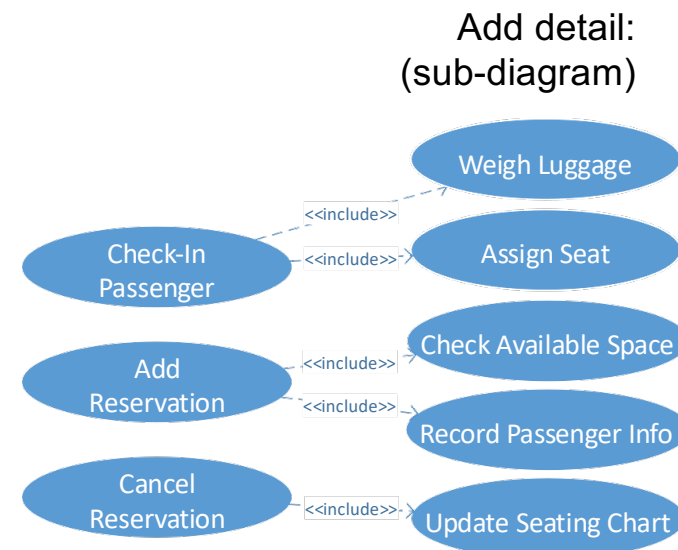
- **Add detail** to the diagram, representing an airline reservation system
- First, create a **separate diagram** (sub-diagram) for the top-level services, and then **add new use cases** that make up the top-level ones.
- View



Example - Airline Reservation System

- There is an `<<include>>` from
 - "Check-In Passenger" to "Weigh Luggage"
- "Luggage" and from
 - "Check-In Passenger" to "Assign Seat"
- This indicates that in order to check in a passenger,
 - luggage must be weighed and
 - a seat must be assigned
- The diagram also indicates that in order to add a reservation to the system,
 - the available space must be checked
 - the passenger's information must be recorded

• .



Example - Airline Reservation System

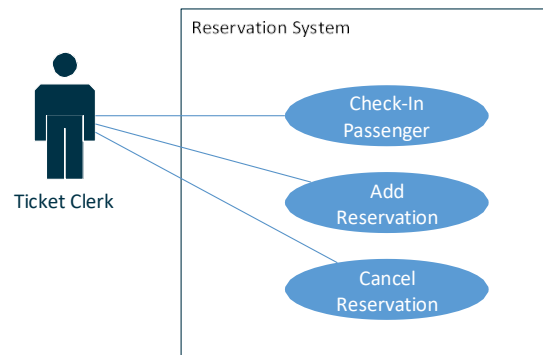


- Special seats anyone?
- Not all of the seats on the airplane are exactly alike (some window and some aisle seats), and sometimes passengers will express a preference for one of these types of seats but not the other.
- But of course, they cannot just be given their preference right away,
- because the seat they want might not be available.
- Therefore, the process of assigning
 - a window seat involves checking for the availability of window seats,
 - an aisle seat involves checking for the availability of aisle seats

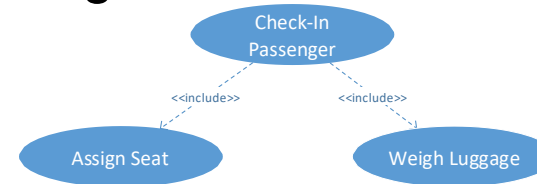
Example - Airline Reservation System



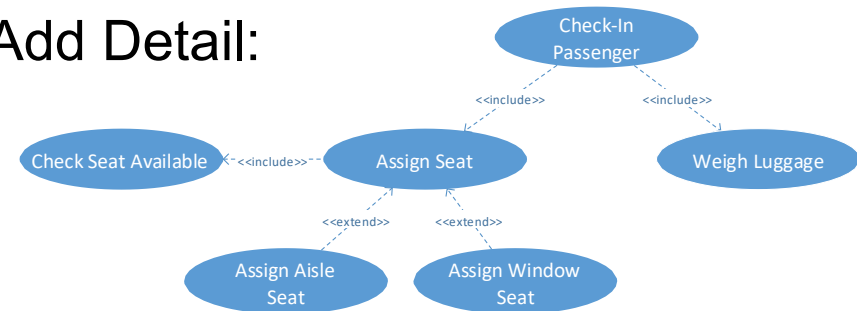
- Initial Design:



- Sub-Diagram:

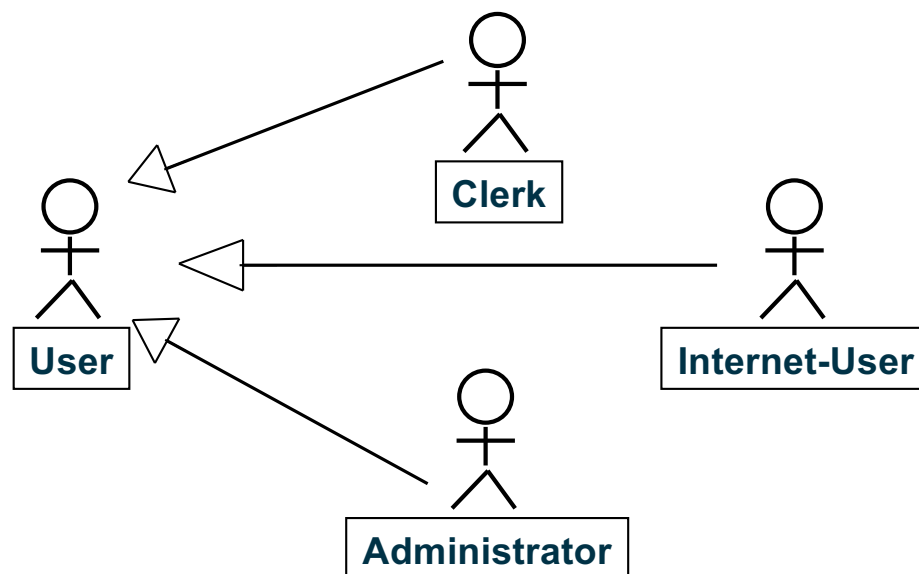


- Add Detail:



Generalization - Specialization

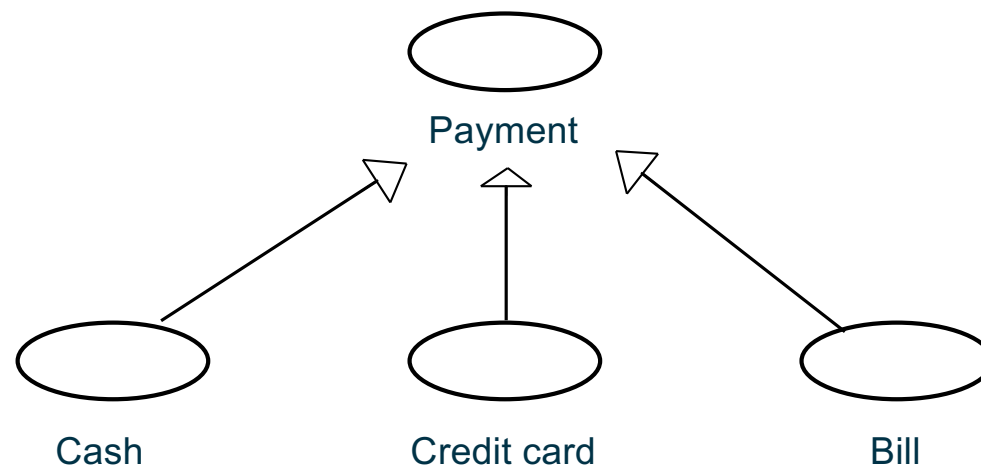
- Actors can be generalized / specialized



Generalization - Specialization

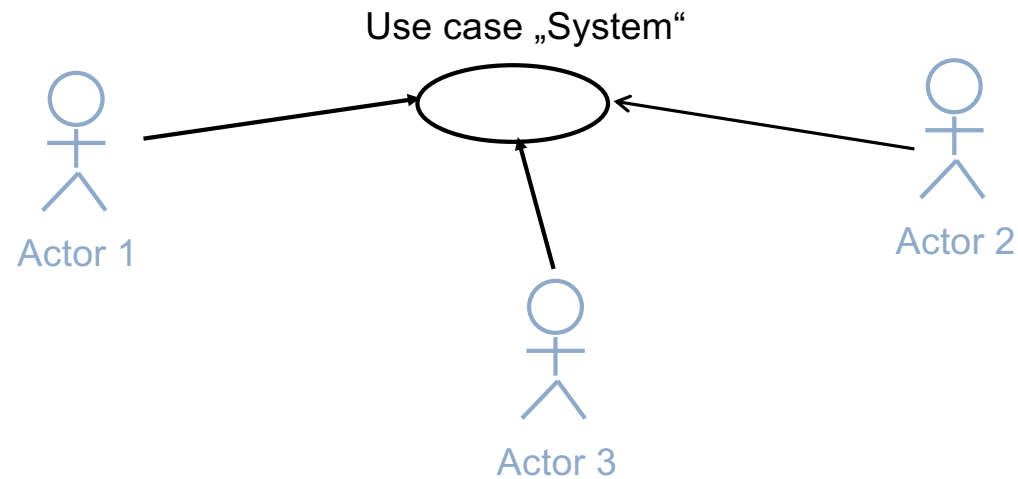


- UCs can be generalized or specialized



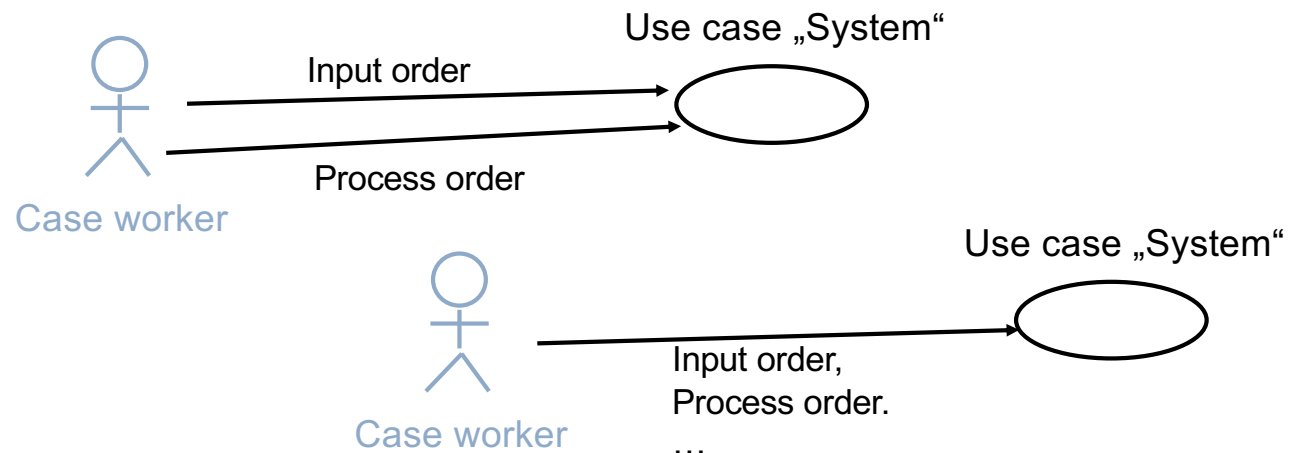
Context Model

- In a context model you draw **one use case for the whole system** and all actors that interact with the system.



Context Model

- If there are many functions an actor can activate in a use case you can either have **many arrows** (one for each function) or **one arrow** representing all functions which then must be documented separately (e.g. in the notes of the relation).



Analysis with use cases



- Requirements analysis with use case diagrams can be done by carrying out the following steps:
 - Draw a context model
 - Write a glossary
 - Detail the context model

For Security: Add Misuse Cases

Security considerations in analysis



Based on the Use-Case terminology in UML

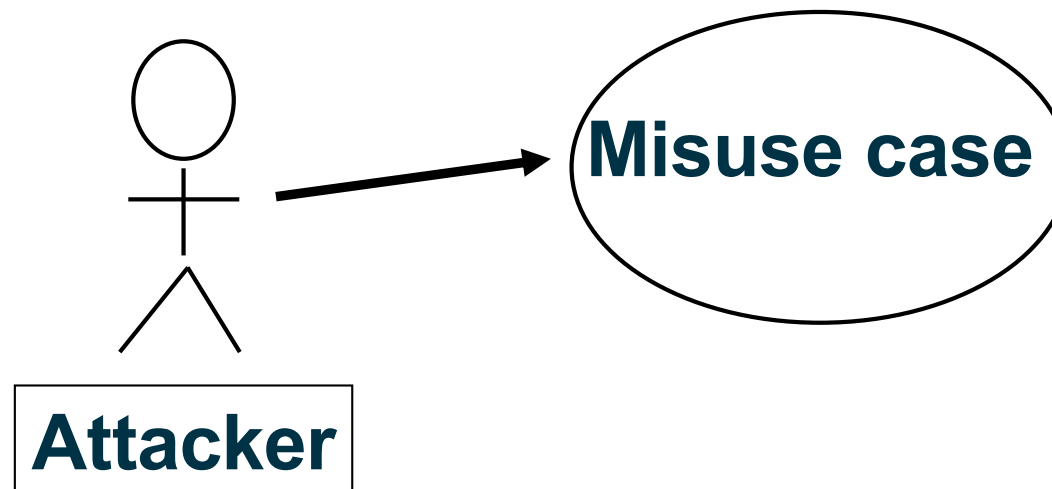
MISUSE CASES

are defined and complemented by

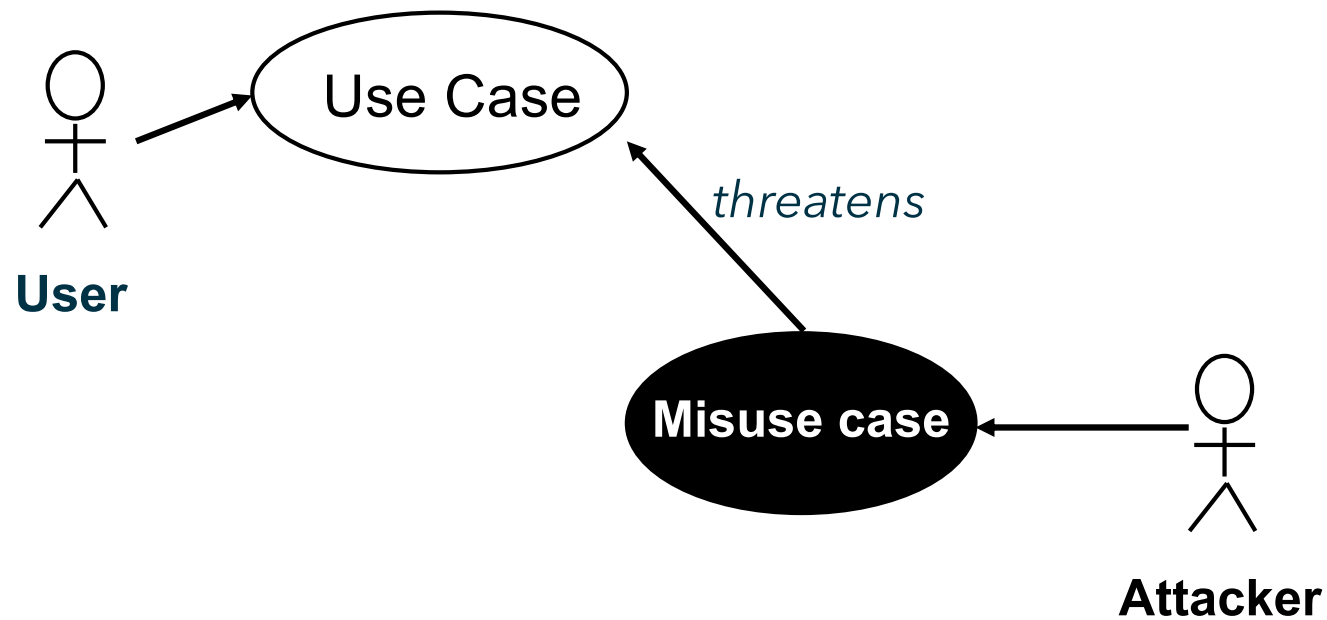
MITIGATION CASES

- Insert MISUSE CASES
- Describe the misuse cases
- Find mitigation use cases

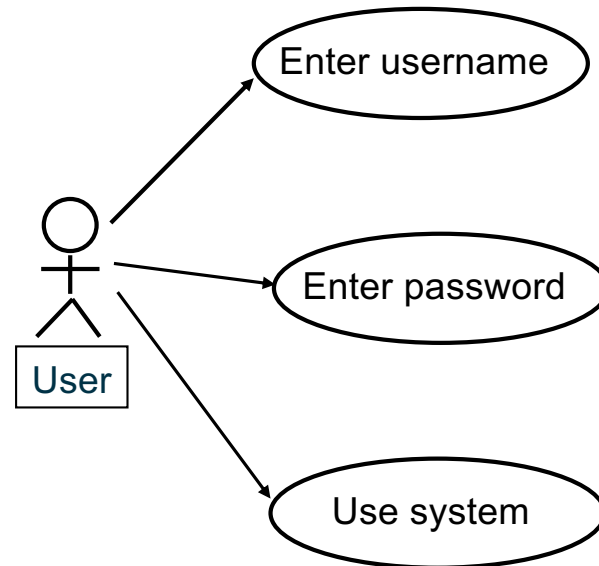
Security considerations in analysis



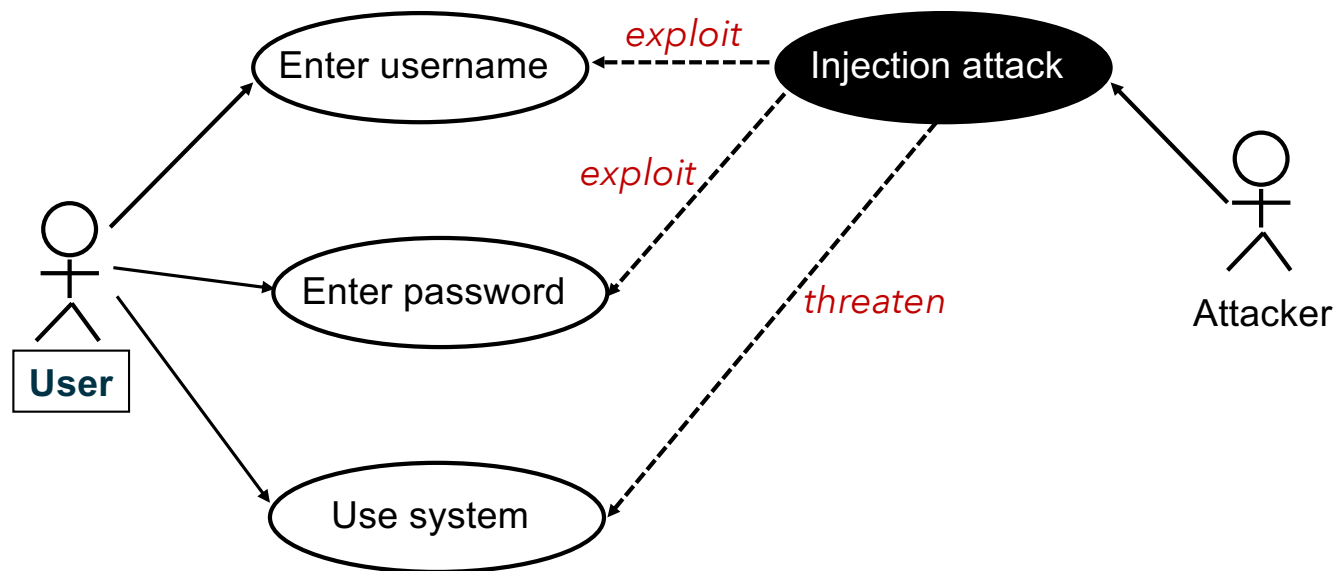
Security considerations in analysis



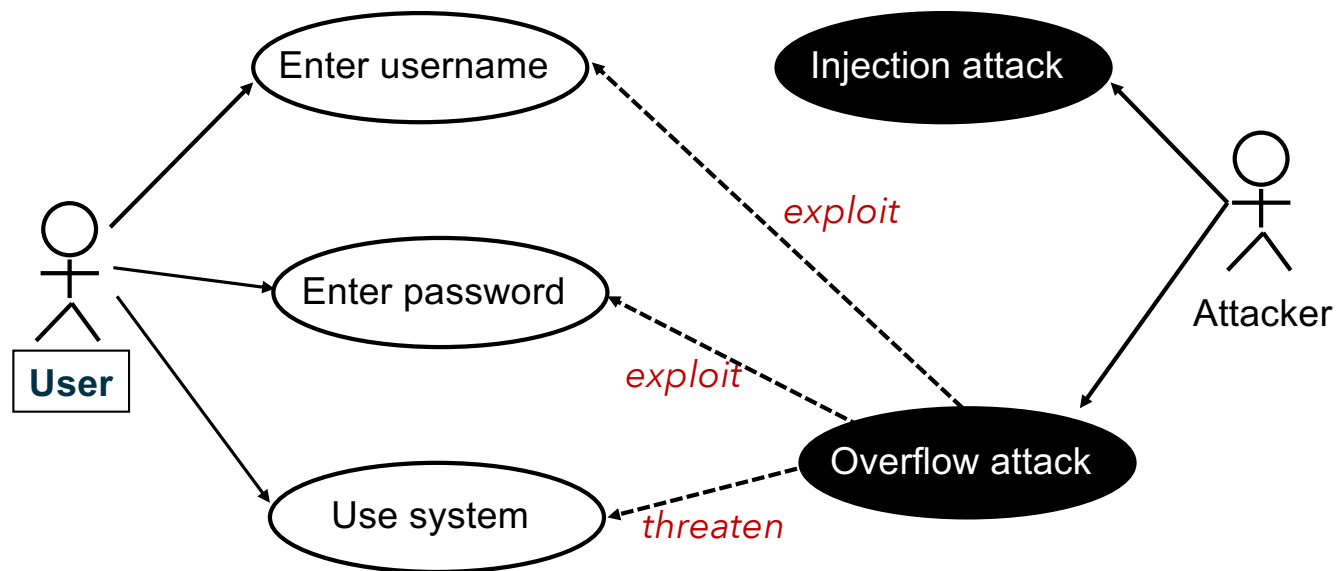
Example: Misuse Cases



Example: Misuse Cases

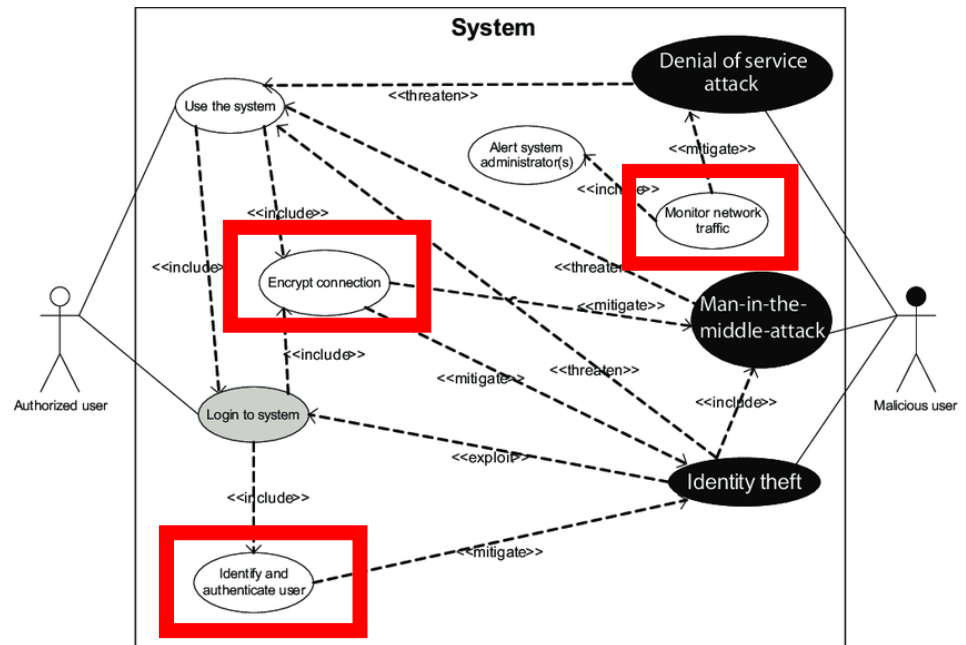


Example: Misuse Cases

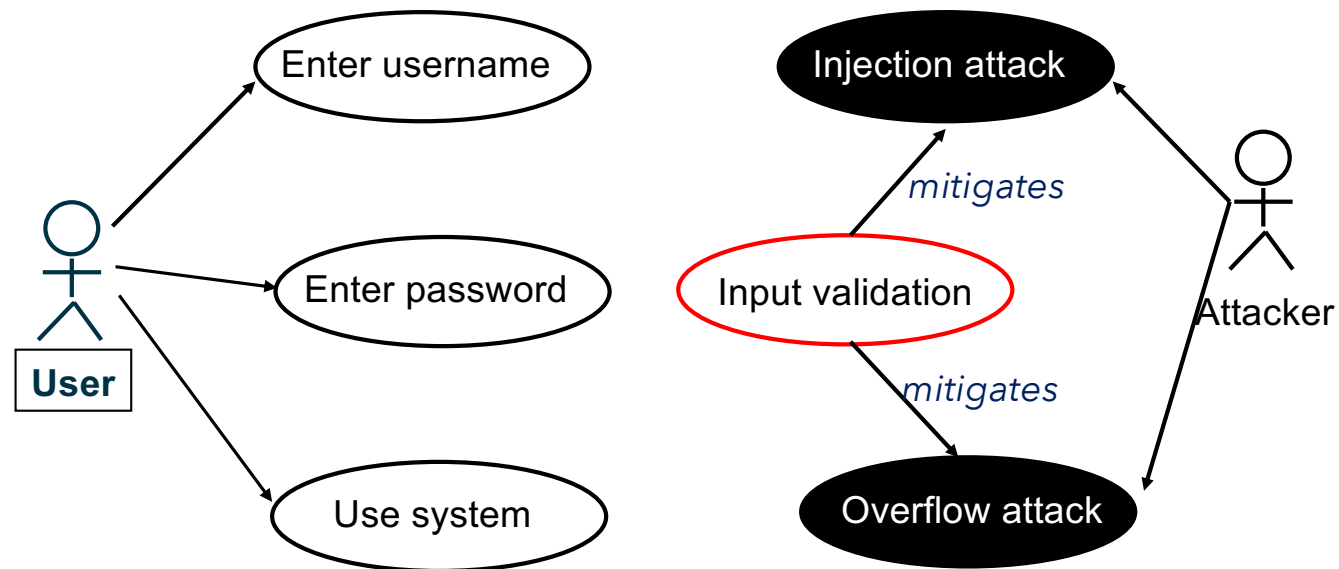


Mitigation

- Insert mitigation use cases
 - that will mitigate the thread of the misuse case
- Describe the mitigation use cases either
 - as part of the misuse case description or
 - as a use case of its own (better)



Example Mitigation Use Cases





Interfaces as Special Problems

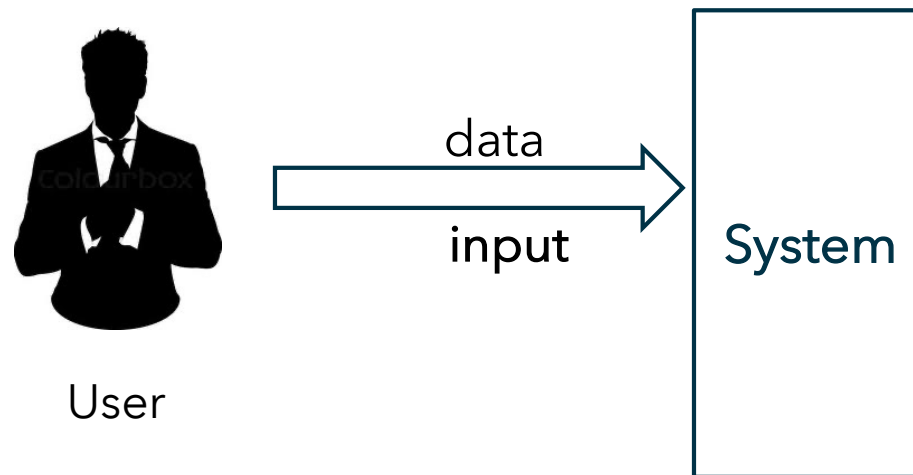
Vulnerabilities - Exploits



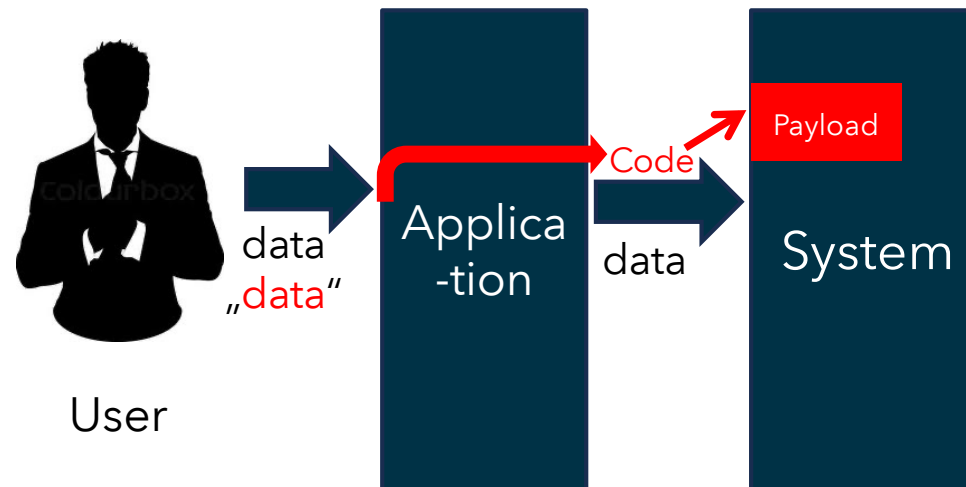
Why are systems vulnerable?

- Systems consist of **components**
 - Components are developed independently
 - Components have input data
 - Components exchange data
- Components **need interfaces**
 - The rules for the input data define a language
 - The language must be defined exactly
 - The language must be interpreted exactly in the same way on both sides of the interface

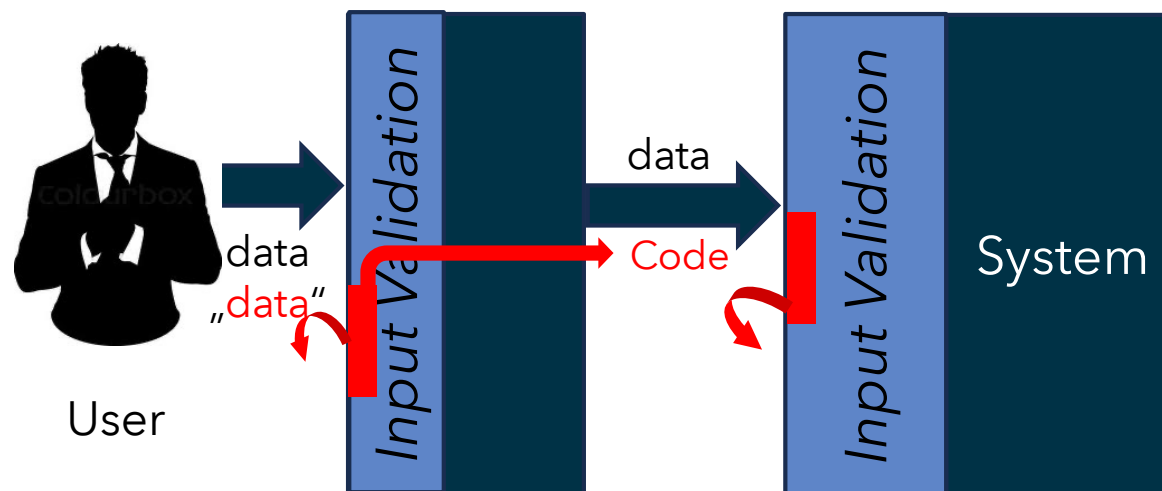
Interfaces - Example



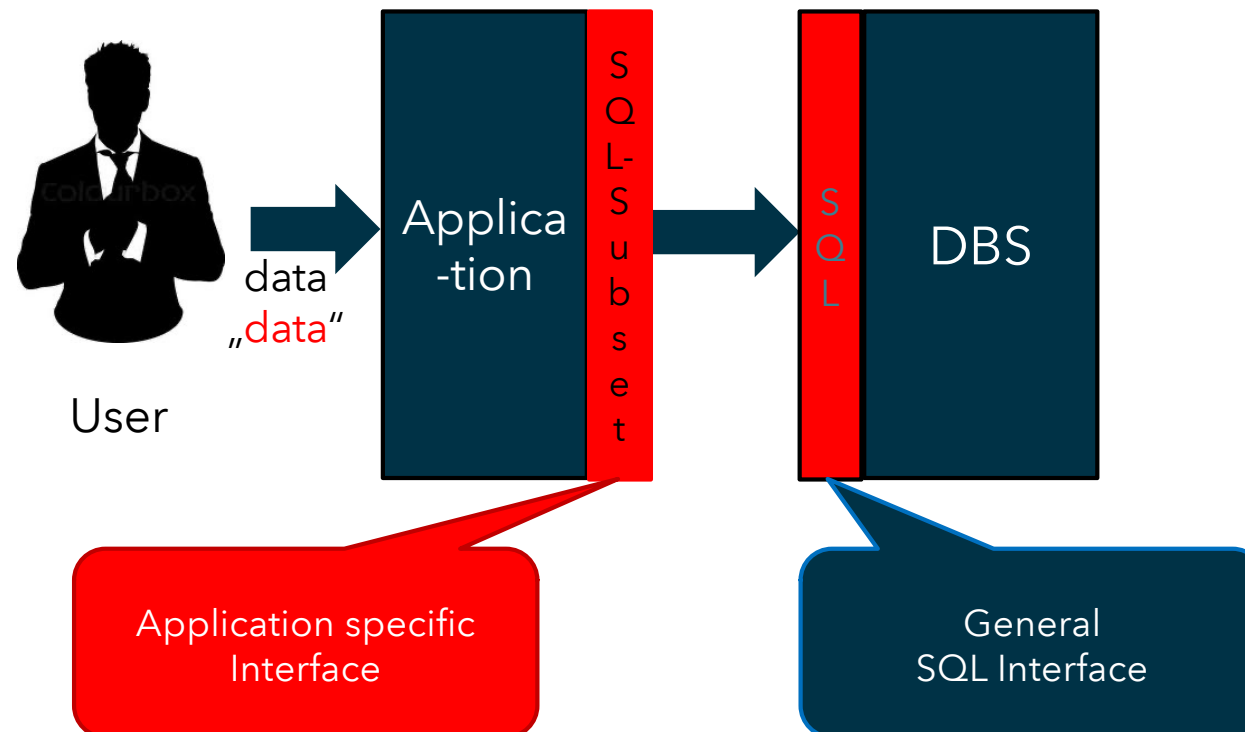
Interfaces - Example



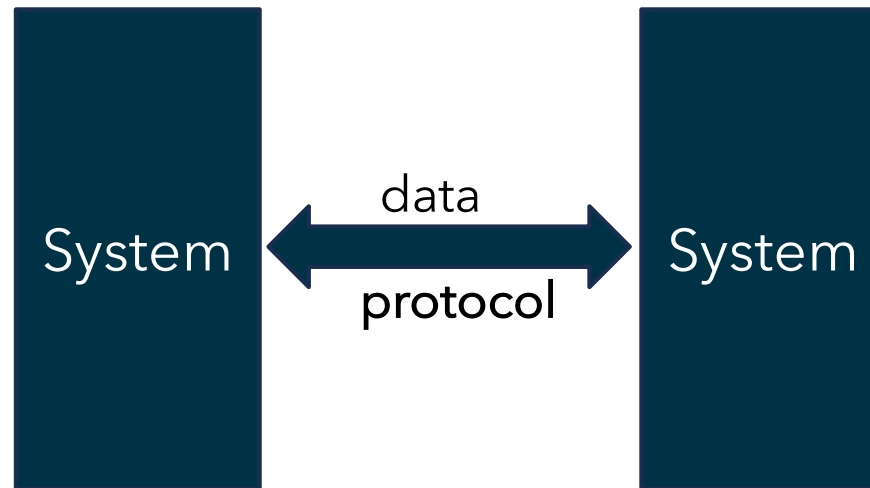
Counter-Example: Input Validation



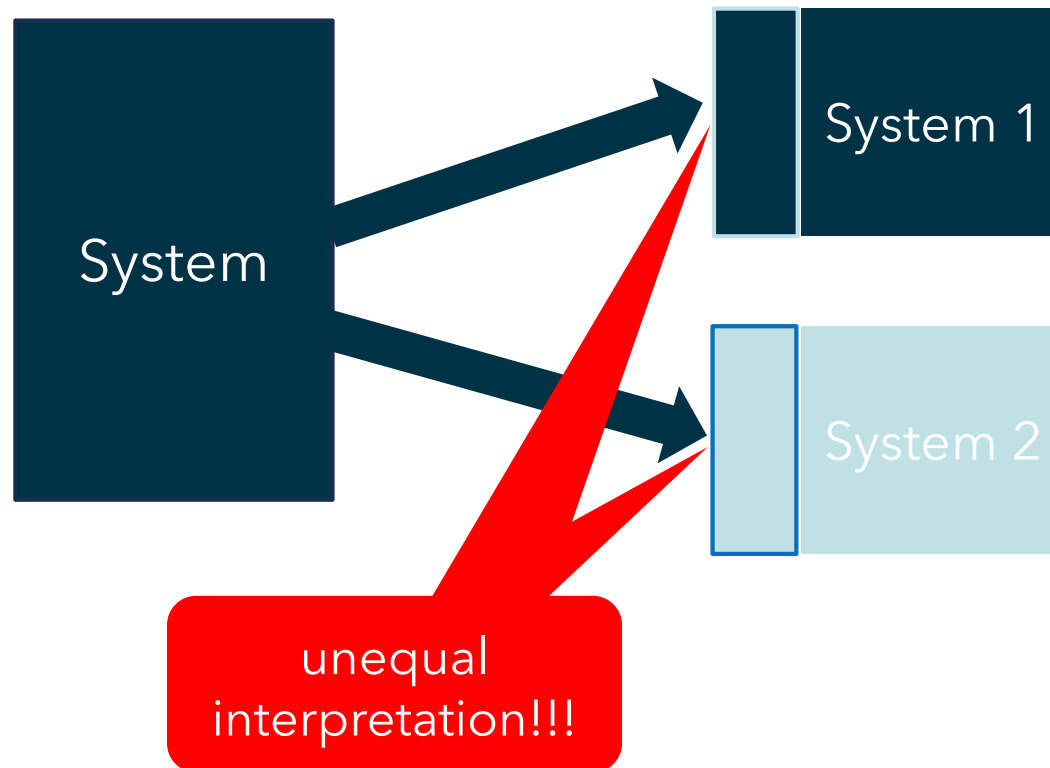
Interfaces - Example



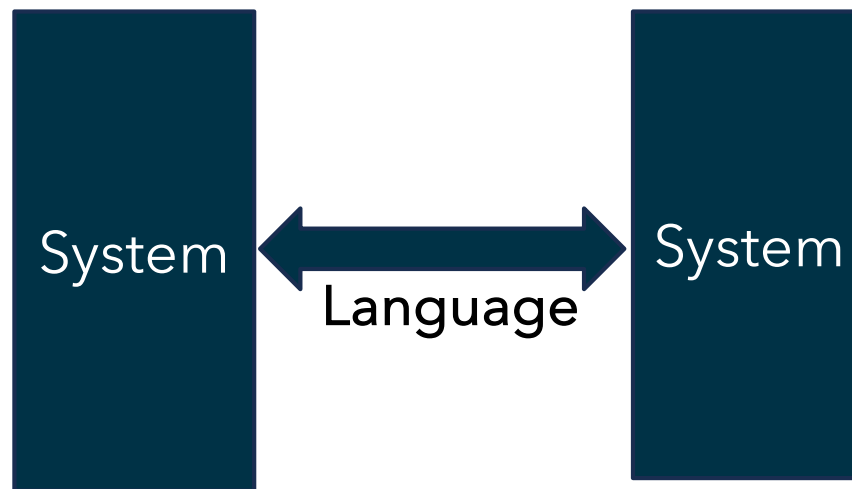
Interfaces - Example



Interfaces - Example



Interfaces



Goal

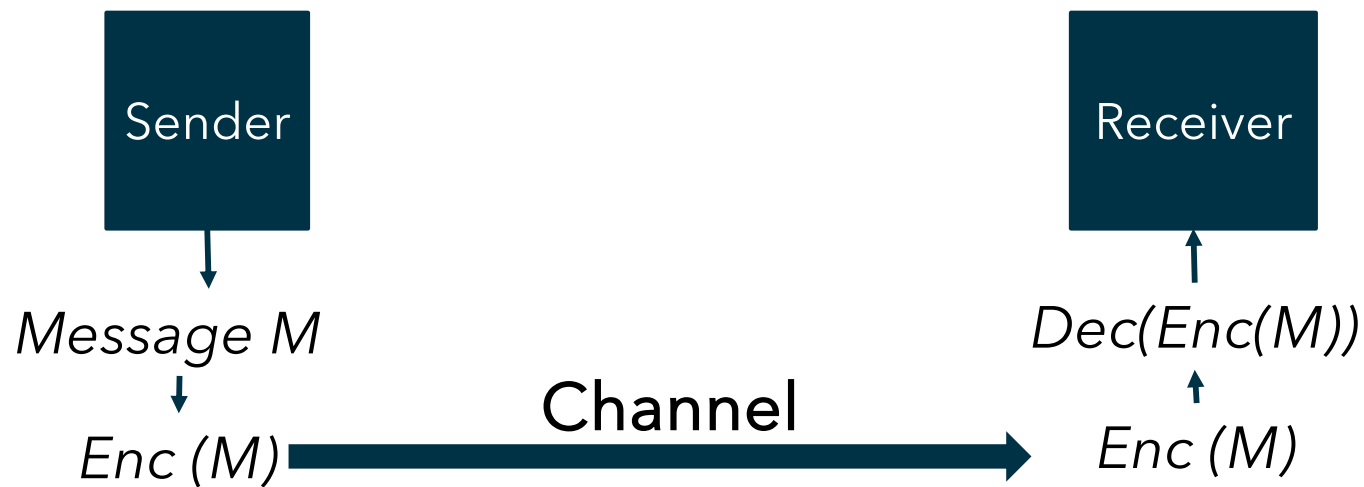


- Secure input validation
 - Accept only valid input
 - Interpret input always in the same way
 - No unforeseen reactions on any input
- Identical interpretation on both sides of the interface
 - The language of the sender must be equal to the language of the receiver
 - Design contracts must be defined explicitly and must be followed in a provable way

Interface



- Communication Model by Shannon, 1948



Interface



There is a massive problem, if:

$$\text{Dec}(\text{Enc}(M)) \neq M$$

→ The receiver reacts differently as expected by the sender!
Reason: the implementation of Dec is wrong!

Interface

Change your view → **Goal of the Attacker**

- Find a combination of M , Enc and Dec with the following properties:
 - $M \neq Dec(Enc(M))$
 - M is semantically valid for the sender
 - $Dec(Enc(M))$ is semantically valid for the receiver
 - The processing of M at the receiver is not equal to the processing of $Dec(Enc(M))$ at the receiver
- Attacker makes use of his own "data"

Typical Exploit-Techniques



Typical techniques used by malware

- SQL Injection
- Command Injection
- XSS
- Buffer Overflows
- String Overflows

See: OWASP Top 10 Threats

https://www.owasp.org/index.php/Category:OWASP_Top_Ten_Project

Check all (input) data

- Which data?
- Where?
- When?

Problems:

- Amount
- Efficiency
- Bypassing
- Streaming

